Convolutional Neural Networks (CNN)

Intelligent Systems

Escuela Politécnica de Ingeniería de Gijón

Universidad de Oviedo

Sandra Ranilla-Cortina

ranillasandra@uniovi.es

Overview

What is a Convolutional Neural Network (CNN)?

What is a Convolutional Neural Network (CNN)?

Convolutional Neural Networks, or **CNNs**, are a class of **deep neural networks** specialized in processing grid-like data (e.g., images), but also text and audio.

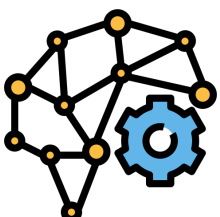
Inspired by human vision, **CNNs** excel at:

→ **Automatic feature extraction**

→ **Reducing parameters**

→ **Pattern recognition**.

Key idea: CNNs automatically learn *what to look for* in an image.



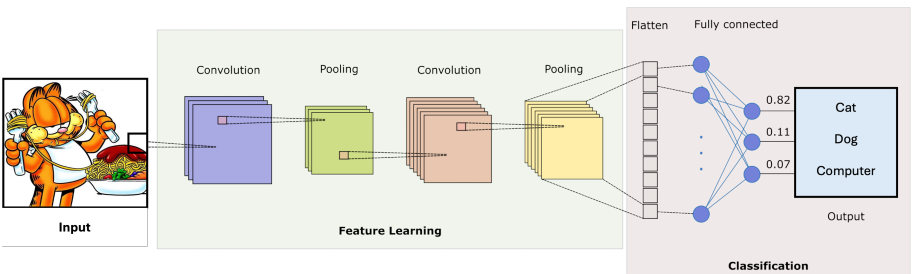
What is a Convolutional Neural Network (CNN)?

CNNs have revolutionized *AI*, achieving state-of-the-art results in tasks like:

→ **Image Recognition**: Facial recognition, medical imaging, self-driving cars

→ **Generative AI**: Deepfakes, artistic style transfer.

Big question: How does a CNN recognize a cat?



From Pixels to Meaning

An image is just numbers

Pixels	$\rightarrow$	**Objects**
Matrix of values (RGB)		edges $\rightarrow$ shapes $\rightarrow$ cat

CNNs learn this transformation automatically using convolutions

Basic Concepts of CNNs

Understanding Filters & Kernels

What is a **filter**?

→ A small matrix (**kernel**) used to detect patterns in an image

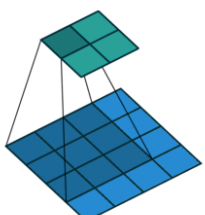
→ Example: Edge detection, sharpening, blurring.

How does it work?

→ The **filter** slides over the image and performs element-wise multiplication

→ Produces a feature map highlighting detected patterns.

Key idea: Each filter detects a specific pattern (e.g., edges)



3x3 filter

Convolution: Step-by-Step

Process:

→ Take a small region of the image

→ Apply the filter (multiply values)

→ Sum the result → one value

Repeat this across the image

Result: A new image (feature map) highlighting patterns

This operation is called *convolution*

Convolution Operation

Mathematical formulation for an input x and kernel w:

$$y\left(i,j\right)=\sum_{m=0}^{M-1}\sum_{n=0}^{N-1}x\left(i+m,j+n\right)\cdot w\left(m,n\right)$$

where:

- $x\left(i,j\right)$ represents the input image pixel value at position (i,j)
- $w\left(m,n\right)$ is the filter (**kernel**) value (**weight**) at position (m,n)
- $y\left(i,j\right)$ is the output feature map at position (i,j).

Padding & Stride in CNNs

The output shape of the convolutional layer is determined by the shape of the input and the shape of the convolution kernel.

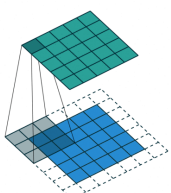
Two important hyper-parameters that affect to the dimensions of y:

Padding

$\rightarrow$ Adds extra pixels around the input to preserve spatial dimensions

$\rightarrow$ Helps preserve spatial dimensions.

Without padding $\rightarrow$ output shrinks



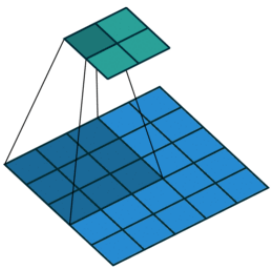
3x3 filter with padding of 1

Padding & Stride in CNN

Stride

→ Controls how much the kernel shifts over the input image

→ Larger stride → smaller output feature map.



3x3 filter with stride of 2

CNN Model Architecture

CNN Model Architecture

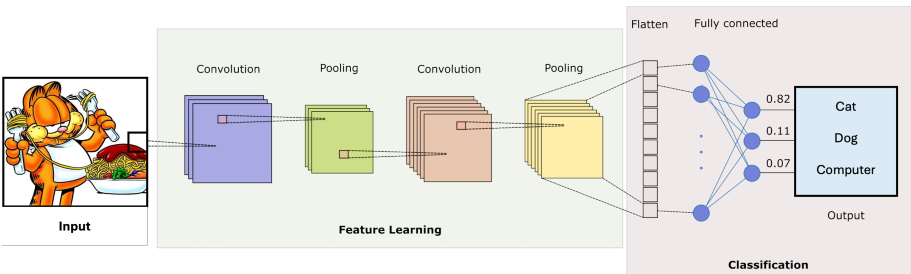
A Convolutional Neural Network (**CNN**) processes data through a sequence of layers:

i) **Input**

ii) **Convolution**

iii) **Pooling**

iv) **Fully Connected**



How data flows through a CNN

A CNN processes data step by step:

→ Input image (raw pixels)

→ Convolution layers extract features (edges, shapes)

→ Pooling reduces spatial size

→ Fully connected layers produce predictions

Pixels → Features → Prediction

Key idea: The network transforms pixels into increasingly abstract representations

Input Layer

The **Input Layer** is where the data is fed into the network:

→ Receives the raw data (e.g., image), and processes it before passing it to the subsequent layers

→ Generally represented as 3D matrices: **height x width x channels** (e.g., 32x32x3 for a color image with 3 channels: red green, and blue)

→ The input data may undergo normalization to improve the network's training efficiency.

Key idea: Input is just raw numerical data

Convolutional Layer

The **Convolutional Layer** is responsible for detecting features from the input data, such as edges, textures, and patterns:

- → **Kernels**: Slide over the input image to detect specific features by computing dot products at each position
- → **Feature Maps**: Generate feature maps that highlight the detected features, such as edges or corners
- → **Padding + Stride**: Control output size and movement of filters
- → **Activation Function**: An activation function (typically **ReLU**) is used to introduce non-linearity into the network.
  ReLU keeps positive values as they are and replaces negative values with zero, helping the network learn complex patterns more efficiently.

Key idea: Extract features from the input

Pooling Layer

The **Pooling Layer** reduces the spatial dimensions of the feature maps, making the network more efficient and helping to prevent overfitting:

→ **Max Pooling**: The most common operation, where the maximum value from a defined window (e.g., 2x2) is selected, preserving important features

→ **Average Pooling**: Instead of the maximum, it takes the average value from the window, though it's used less frequently.

Key idea: Reduce size while keeping important information

Fully Connected Layer

The **Fully Connected Layer** is where the network makes final predictions based on the features extracted by the previous layers:

- → Each neuron in the **fully connected layer** is connected to every neuron in the previous layer, forming a **dense network**
- → The output from the previous layers is **flattened** (i.e., converted into a 1D vector) before being passed into the FC layer
- → Combines extracted features to produce final predictions
- → Typically uses **sigmoid** (binary) or **softmax** (multi-class)

Key idea: Make the final decision

Exercise: Understanding CNN Flow

Given a CNN architecture:

Input (32x32x3) $\rightarrow$ Conv $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ FC $\rightarrow$ Output

Questions:

- $\rightarrow$ Which layers extract features?
- $\rightarrow$ Which layers reduce spatial dimensions?
- $\rightarrow$ Where does the final decision happen?
- $\rightarrow$ How does the data change through the network?

Work in pairs (10-20 min)

Solution: Understanding CNN Flow

Architecture:

Input (32x32x3) → Conv → Pool → Conv → FC → Output

→ **Feature extraction**: Convolutional layers

→ **Dimensionality reduction**: Pooling layer

→ **Final decision**: Fully Connected layer

→ **Data transformation**:

- Spatial size decreases
- Feature depth increases
- Final flattening → classification

CNN Model Training

What does Training mean?

Goal: Learn from data

→ The model makes predictions

→ Compares them with true labels

→ Adjusts its parameters to reduce errors

This process is repeated many times

Key idea: Training = learning from mistakes through iteration

CNN Model Training

Training a **CNN** means optimizing its parameters to improve predictions.

Key steps:

i) **Forward Propagation**: Input $\rightarrow$ Predictions

ii) **Loss**: Compare predictions with true labels

iii) **Backpropagation**: Compute gradients

iv) **Optimization**: Update weights (Gradient Descent)

v) **Validation**: Evaluate on unseen data (prevent overfitting)

vi) **Evaluation**: Final performance metrics (Accuracy, MSE, etc.)

Parameters in CNN Model Training

Different **parameters** and **hyperparameters** are used during training:

i) **Forward Propagation**:
   - **Layer hyperparameters**: filters (size/number), padding, stride, units (FC)

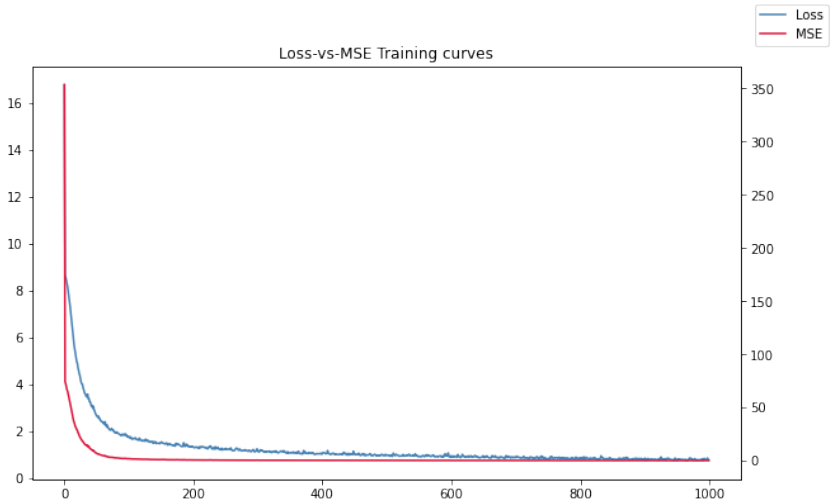
ii) **Loss Calculation**:
   - **Loss function**: Cross-Entropy (classification), MSE (regression)

iii) **Backpropagation & Optimization**:
   - **Trainable parameters**: weights (CONV, FC)
   - **Optimizer**: SGD, Adam, etc.
   - **Hyperparameters**: learning rate, momentum

iv) **Iterations**:
   - **Batch size**: samples per update
   - **Epochs**: number of training passes

CNN Training: Loss & Performance Curves over Epochs

Training: Loss ↓, Performance ↑, possible overfitting



Training Performance of CNN Model

Applications of CNNs

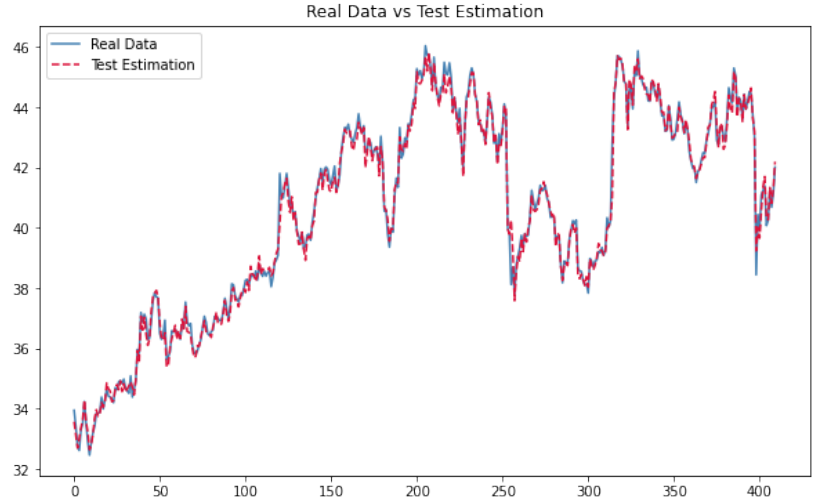
Applications of CNNs

Convolutional Neural Networks (**CNNs**) have a wide range of applications:

- → **Image Classification**: medical imaging, autonomous vehicles
- → **Object Detection**: facial recognition, security systems
- → **Segmentation**: medical image analysis, satellite imaging
- → **NLP**: sentiment analysis, voice assistants
- → **Time Series Forecasting**: stock prices, weather
- → **Video Analysis**: surveillance, autonomous driving
- → **Generative Models**: style transfer, super-resolution
- → **Anomaly Detection**: fault detection, fraud

Key idea: CNNs are versatile pattern-learning models

Time Series Forecasting - Stock prices



CNN Model - Stock Market Price Estimation

Generative Models - Style Transfer

CNN Model - Style Transfer

NLP - Sentiment analysis

CNN Model - Sentiment Analysis

Exercise: Where would you use CNNs?

Consider the following problems:

→ Detecting tumors in medical images

→ Predicting stock prices

→ Classifying emails as spam or not

→ Recognizing objects in a video

Questions:

→ Would you use a CNN? Why or why not?

→ What kind of patterns would it learn?

Discuss in groups (10-20 min)

Solution: Where would you use CNNs?

→ **Medical images**: YES → spatial patterns (shapes, anomalies)

→ **Stock prices**: SOMETIMES → local temporal patterns

→ **Spam emails**: NOT typical → better with NLP models

→ **Video**: YES → image-based features over time

Key idea:

CNNs work best with spatial or locally structured data

Conclusions

Final Perspective

Main ideas:

→ Images are just **numbers**

→ CNNs learn to extract **meaningful patterns**

→ These patterns allow models to **understand data**.

From pixels → patterns → understanding

Key message: Learning representations is at the core of modern AI

References

References i

[1] Bishop, C. M., and Nasser, M. N. *Pattern recognition and machine learning*. New York: springer, 2006.

[2] Charu, C. A. *Neural networks and deep learning*. Cham: Springer, 2018.

[3] Chollet, F. *Deep learning with Python*. Simon and Schuster, 2021.

[4] Deisenroth, M. P., Faisal, A. A., and Ong, C. S. *Mathematics for machine learning*. Cambridge University Press, 2020.

[5] Goodfellow, I., Bengio, Y., Courville, A., and Bengio, Y. *Deep learning*. Cambridge: MIT press, 2016.

[6] Géron, A. *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems*. O'Reilly Media, Inc., 2022.

[7] López de Prado, M. *Advances in Financial Machine Learning*. John Wiley & Sons, 2018.

[8] Raschka, S., and Mirjalili, V. *Python machine learning: Machine learning and deep learning with Python, , scikit-learn, and TensorFlow 2*. Packt publishing ltd, 2019.